
GFTEvolver

Introductory User Guide

Adam K. Thomasson

9 August 2026

GFTEvolver developed by Steven J. Cotterill and Adam K. Thomasson.

Document scope

This is a short starting guide for students. It explains how to compile and run the code, how the supplied configuration files fit together, where output is written, and how to begin implementing a position-dependent field configuration initial condition.

Contents

1	Compiling and running	2
2	How the configuration system works	2
3	The lattice configuration	3
4	Selecting the physical model	4
5	Initial conditions	6
6	Analysers and output files	7
7	A first-run checklist	7

1 Compiling and running

1.1 Requirements

The program requires:

- a C++17 compiler;
- CMake 3.10 or newer;
- Ninja if the commands below are used unchanged;
- an MPI implementation with C++ support only for a parallel build.

On Linux, a typical required installation is:

```
sudo apt update
sudo apt install build-essential cmake ninja-build
sudo apt install libopenmpi-dev openmpi-bin    # MPI build only
```

1.2 Serial build and run

From the top-level source directory:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/GFTEvolver
```

1.3 MPI build and run

Use a separate build directory if you'd like to compare serial and MPI runs:

```
cmake -S . -B build-mpi -G Ninja -DCMAKE_BUILD_TYPE=Release -DGFT_ENABLE_MPI=ON
cmake --build build-mpi
mpirun -np 4 ./build-mpi/GFTEvolver
```

MPI divides the global lattice into slabs along x . The number of ranks need not divide n_x exactly, but each rank must own enough x -planes for the numerical stencil. If the code reports that a slab is too narrow, use fewer ranks or a larger n_x .

When to rebuild

Editing a `.cfg` value does not require recompilation. Editing C++ source or headers does. The executable contains the absolute source path used by CMake, so rerun CMake and rebuild if the complete source directory is moved.

1.4 Where output goes

`Data/` and `Data/Continual/` are created beside the source automatically. A run warns if `Data/` is already non-empty, because files with the same configured names may be overwritten. Move or rename results that must be retained before starting another run.

2 How the configuration system works

Every active configuration file is read positionally. Descriptive text is skipped and values are read after successive colons. Therefore, for ordinary use:

- change values to the right of the colon only;
- use 0/1 for false/true questions;
- preserve the order, colons and introductory line structure;
- avoid whitespace in configured filenames.

The start-up text prints the values that were actually read. Check it before trusting a run.

Configuration	Purpose
Core/Config/Lattice.cfg	Grid, timesteps, flow/dynamics, component counts, IC selection, analysers and boundary faces.
Core/Config/Model.cfg	Potential/derivative/Wilson-loop selection and dynamical damping.
Potentials/Config/	Parameters for the selected scalar potential.
Gradients/Config/	Numerical thresholds for the selected covariant derivative.
WilsonLoops/Config/	Gauge couplings, representation and numerical threshold.
InitialConditions/Config/	Random or user-profile IC settings.
Analysers/Config/	What is written, how often, and under which filename.

Only the configuration belonging to a selected model, IC or analyser is constructed and used.

3 The lattice configuration

3.1 Grid and evolution phases

Entry in Lattice.cfg	Meaning
nx ny nz	Numbers of global physical lattice sites. MPI decomposes only nx .
dx dy dz	Physical lattice spacings. The box lengths are $n_x d_x$, $n_y d_y$ and $n_z d_z$.
nt	Number of full dynamical timesteps, unless the light-crossing override is enabled.
dt	Dynamical timestep. It must be finite and positive. The code does not choose a stable value automatically.
Initial gradient flow	If 1, relaxes the IC before dynamics using step dTau .
Flow timesteps	Fixed number of gradient flow steps, or maximum number when residual stopping is enabled.
Residual stopping	Stops when both global maximum scalar and gauge EOM residuals are at or below their separate tolerances.
Full dynamical evolution	If 0, no physical leapfrog evolution follows the optional gradient flow phase.
Light-crossing override	If enabled during dynamics, replaces nt by $\lfloor N_{lc} n_x d_x / (2dt) \rfloor$, where N_{lc} is the following entry defining the number of light-crossing times.

Gradient flow uses **dTau**, has no dynamical friction term, and begins with the two stored time buffers synchronised. When gradient flow ends, the relaxed configuration is copied into both buffers before dynamics starts. Consequently, an intentionally non-zero initial velocity is erased whenever initial gradient flow is enabled.

3.2 Field counts and initial-condition selectors

Integer number of scalar field components is the total number stored at each site. The gauge-field entry is the number stored for *one* link direction; the code multiplies it by three for the *x*-, *y*- and *z*-directed links.

The two IC selectors are ordered **scalar gauge**:

Value	Type	Behaviour
0	Zero	Scalars remain zero. Gauge links remain trivial: zero generator coordinates or identity SU(2) quaternions.
1	Random uniform	Uses <code>RandomUniform.cfg</code> . Normally used for scalar fields; arbitrary random quaternion components are not valid SU(2) links.
2	User profile	Calls the profile template described in Section 5. The untouched template retains the representation-safe defaults.

Examples are 1 0 for random scalars and trivial gauge links, 2 0 for a user scalar profile, and 2 2 when both scalar and gauge profiles have been implemented.

3.3 Analysers, boundaries and terminal reports

The three analyser switches activate `OutputFields.cfg`, `Energy.cfg` and `GaugeCondition.cfg`, respectively. A disabled analyser does not use its own configuration file.

Boundary conditions are entered for the six faces in the order

$$(-x), (+x), (-y), (+y), (-z), (+z).$$

Value	Boundary	Meaning
1	Fixed	Boundary support sites retain their initial field values and are not evolved. Fixed support currently assumes stencil radius one.
2	Neumann	Reserved but not implemented; do not select it.
3	Periodic	Opposite faces of the same axis must both be periodic.

Edges and corners are inferred automatically from the face assignments. MPI internal x -interfaces are also assigned automatically; users must never enter the internal code value 4. Mixed choices such as periodic x with fixed y, z are allowed.

The final two entries enable terminal progress messages and set their frequency. Any enabled report or continual output frequency must be a positive integer.

4 Selecting the physical model

4.1 The three selectors in `Model.cfg`

Selector	Values	Meaning
Potential	0, 1, 2	Null, $SO(N)$, or $SO(N) \times SO(M)$ (<code>Double_S0_N</code>).
Derivative	0, 1, 2, 3	Null, ordinary global derivative, SM covariant derivative, or 2HDM covariant derivative.
Wilson loop	0, 1, 2	Null gauge dynamics, SM gauge dynamics, or 2HDM gauge dynamics.

The derivative and Wilson-loop choices should normally describe the same gauge theory. The main supported field layouts are:

Theory	Scalars	Derivative	Wilson loop	Gauge entries per direction
Global scalar theory	Any	1	0	Usually 0
SM gauge theory	4	2	1	4 generator; 5 quaternion
2HDM with SM gauge group	8	3	2	4 generator; 5 quaternion
2HDM plus family $U(1)$	8	3	2	5 generator; 6 quaternion
2HDM plus family $SU(2)$	8	3	2	7 generator; 9 quaternion

For the 2HDM Wilson-loop class, **Is the generator representation...** must agree with the chosen gauge count. For the SM Wilson-loop class the representation is inferred: four entries per direction means generator coordinates and five means a hypercharge coordinate plus an $SU(2)$ quaternion.

4.2 Dynamical damping

The damping factor in `Model.cfg` is the magnitude of the artificial friction term applied at the start of *dynamical* evolution. It acts for the specified number of dynamical timesteps. Gradient-flow steps are separate and do not consume the dynamical damping count.

4.3 Potential configuration

`S0_N.cfg` defines $V = \frac{\lambda}{4} (|f|^2 - \eta^2)^2$ using `lambda` and `etaSqr`. The number of components comes from `Lattice.cfg`.

`Double_S0_N.cfg` splits the scalar array into f_1 , containing the first configured number of components, and f_2 , containing the remainder. Its parameter groups are:

- `m1,m2,l1,l2,l3`: the $SO(N) \times SO(M)$ -symmetric magnitude terms;
- `m12r,l4p5,l6r,l7r`: dot-product terms enabled by the single- $SO(N)$ switch;
- `m12i,l4m5,l5i,l6i,l7i`: antisymmetric-product terms enabled by the further $U(N/2)$ switch.

The full equations and sign conventions are written at the top of the configuration file. Single- $SO(N)$ terms require equal f_1, f_2 component counts. The $U(N/2)$ terms additionally require that common count to be even. The optional potential-energy offset adds a constant to the reported potential energy; it does not change the equations of motion.

4.4 Gradient and Wilson-loop configuration

The SM and 2HDM gradient configuration files contain a small numerical tolerance used when a group-coordinate magnitude would otherwise cause division by zero. It is not a physical coupling.

The Wilson-loop configurations contain g , g' , and, for the extended 2HDM class, g'' . These are the isospin, hypercharge and family gauge couplings. The Wilson-loop division threshold prevents numerical division by extremely small squared couplings or group-coordinate magnitudes. Change it only with a numerical reason.

5 Initial conditions

5.1 Random uniform initial conditions

`InitialConditions/Config/RandomUniform.cfg` contains:

- the random seed;
- the minimum and maximum of the uniform distribution;
- whether the previous and current buffers are identical.

Equal buffers correspond to zero initial finite-difference velocity. The implementation advances through one global random sequence, so a fixed seed produces the same owned physical field independently of the MPI slab decomposition.

5.2 Implementing a user-defined profile

The supplied class deliberately separates the editable physics from the MPI-safe lattice loop. The relevant files are:

- `InitialConditions/Source/UserDefinedProfile.cpp`;
- `InitialConditions/Header/UserDefinedProfile.hpp`;
- `InitialConditions/Config/UserDefinedProfile.cfg`.

The usual workflow is:

1. Select IC type 2 for the required scalar and/or gauge field in `Lattice.cfg`.
2. Implement `setScalarProfile()` and, only if required, `setGaugeProfile()` in the labelled user sections.
3. Add parameters or loaded data as class members in the header. Add matching config entries and matching reads in `configure()`.
4. Load a small tabulated profile once in `configure()`, not inside a function called at every site. Remove the blank-template warning after the selected profile is genuinely implemented, then rebuild.

The profile functions receive global lattice indices. A centred coordinate may be constructed as

```
const double x = (static_cast<double>(global_x)
    - 0.5*static_cast<double>(geometry.globalNx - 1U))*geometry.dx;
```

Use the same pattern for y, z . Assign `current_fields[component]` according to the model's component order. For zero initial velocity, finish by copying every current component into the previous buffer:

```
for (unsigned comp = 0; comp < num_components; comp++)
{
    previous_fields[comp] = current_fields[comp];
}
```

For gauge fields, `num_components` includes all three link directions and

```
const unsigned direction_width = num_components/3U;
current_fields[dir*direction_width + component]
```

selects one component of direction `dir = 0,1,2`. In generator representation, zeros are trivial links. In quaternion representation, each $SU(2)$ factor is stored as (c^0, c^1, c^2, c^3) with

$$(c^0)^2 + (c^1)^2 + (c^2)^2 + (c^3)^2 = 1.$$

The blank template has already been initialised with identity quaternions $(1, 0, 0, 0)$. Do not replace them with four zeros. Non-zero gauge electric data are group dependent and should not be created by a naive component-wise difference between the two buffers.

What to probably leave alone

`setInitialFields()` already loops only over physical sites owned by the current MPI rank, maps local x to global x , and leaves halo population to the existing exchange. A user profile shouldn't require any edits to this function, the lattice, the boundaries, or the MPI code.

6 Analysers and output files

6.1 Field output

`OutputFields.cfg` controls initial, final and continual field snapshots. If `Output both timesteps` is false, one physical time level is written; if true, the previous complete lattice is followed by the current complete lattice. Each data row contains all scalar components followed by all stored gauge components at one site. Sites are ordered with z varying fastest, then y , then x . MPI rank files are merged into global x -order at the end of the run.

All configured paths are relative to `Data/`. The continual basename is extended by the completed combined evolution-step number.

6.2 Energy and gauge constraints

`Energy.cfg` can write total, potential, gradient, kinetic, magnetic and electric energy. Global quantities are volume-integrated over evolved sites. Local options write the corresponding density arrays.

`GaugeCondition.cfg` can write the spatial integral of the absolute Gauss-constraint violation, its maximum absolute value, and optional local values. Global file entries follow the enabled config order: all integrated constraints, then all maxima.

Only enabled quantities are written, and the plain data files contain values rather than a time column. Record `dt` and the output frequency with each run.

Output cost

Global diagnostics are inexpensive and are the best default. Field snapshots and local arrays scale with the full lattice volume and can dominate storage and output time.

7 A first-run checklist

Before increasing the production size, confirm that:

1. the start-up summary matches the intended selectors, counts, couplings, boundaries and output paths;
2. the scalar and per-direction gauge counts match the table in Section 5;
3. periodic boundaries occur in opposite pairs;
4. all enabled report and output frequencies are positive;
5. energies and gauge residuals behave sensibly for the problem;
6. a serial and small MPI run agree before using many ranks;
7. the result is stable under reductions of `dt` and lattice spacings.

Message or symptom	First check
Invalid scalar/vector component count	Make <code>Lattice.cfg</code> , derivative, Wilson loop and gauge representation consistent.
Periodic-pair error	Set both faces of that axis to 3, or neither.
MPI slab too narrow	Reduce the number of ranks or increase <code>nx</code> .
Blank user-profile warning	Expected until the selected user function is implemented and the warning is removed.
Existing-output warning	Preserve the old <code>Data/</code> contents before running with the same filenames.

GFTEvolver is research code: a successful run is a software check, not by itself a validation of the physical setup. Keep configuration copies with results and perform the usual timestep, spacing, volume and boundary-condition checks for the calculation being studied.